Ollama LLM Model Recommendations Report

Date: 2026-07-21

Project: MonkeyLM

Hardware Profile

Component	Detail
GPU	NVIDIA GB10 (Grace-Blackwell, unified memory architecture)
VRAM	Shared with system RAM (unified)
System RAM	121 GB total, ~98 GB available
CPU	20 cores (10x Cortex-X925 + 10x Cortex-A725)
Disk	3.6 TB (321 GB free)
Feasible Model Size	Up to ~90 GB comfortably

This is a high-end ARM-based system capable of running even 70B-class models at Q8 quantizations.

Current Configuration (from `monkeylm/config.py`)

Variable	Current Default	Type	Status
`OLLAMA_MODEL`	`minimax-m3:cloud`	Cloud-only	**Not local**
`PDF_VISION_MODEL`	`llama3.2-vision`	Local Ollama	Installed
`VISION_MODEL`	`gemini-3-flash-preview`	Cloud API	Not Ollama

---

1. OLLAMA_MODEL -- Action Planning / Tool Calling

Use case: The primary decision-making LLM. Must emit valid JSON tool calls reliably, reason across multi-step agent loops, and follow structured action schemas. Configured with `temperature=0.2`, `top_p=0.9`, `num_ctx=4096`.

Top Recommendations (Researched)

Rank	Model	Size	VRAM (Q4)	Tool-Call Reliability	Rationale
1	`qwen3.6:35b-a3b-mtp-q8_0`	35B MoE (3B active)	~38 GB	Strong (qwen3_coder parser)	Current SOTA for agentic loops. Native tool calling with dedicated parser. MoE architecture = fast inference. Apache 2.0.
2	`qwen3-coder:30b`	30B MoE (3.3B active)	~19 GB	Strong	Best quality-per-VRAM coder. 256K context. Trained with long-horizon RL on SWE-Bench tasks.
3	`qwen3.6:27b-q8_0`	27B dense	~29 GB	Strong	Dense alternative to MoE. 262K context. SWE-Bench Verified 77.2.
4	`gemma4:31b`	31B dense	~19 GB	Good	Native function calling + structured JSON. Apache 2.0.
5	`hermes3:8b-llama3.1-q8_0`	8B dense	~8.5 GB	Good	Lightweight fallback. Steerable reasoning.

Already Installed (Relevant)

- `qwen3.6:35b-a3b-mtp-q8_0` (38 GB) -- **installed, top pick**
- `qwen3.6:27b-mtp-q8_0` (29 GB) -- installed
- `qwen3-coder:latest` (18 GB) -- installed
- `gemma4:31b` (19 GB) -- installed
- `hermes3:8b-llama3.1-q8_0` (8.5 GB) -- installed
- `qwen3:8b` (5.2 GB) -- installed
- `llama3-groq-tool-use:latest` (4.7 GB) -- installed (BFCL 89.06%)

Recommendation

Primary: `qwen3.6:35b-a3b-mtp-q8_0` -- Already installed. The Qwen 3.6 family ships with a dedicated `qwen3_coder` tool-call parser, closing the historical gap to cloud APIs. The 35B-A3B MoE activates only ~3B parameters per token, giving it the reasoning quality of a large model at near-small-model speed. Community reports indicate better edge-case handling on nested JSON arguments and missing-parameter errors than the 2.5 line.

Fallback: `qwen3-coder:30b` -- Already installed. If the 35B-A3B proves too heavy or has tool-call formatting issues, this is the best quality-per-VRAM alternative with 256K context.

Lightweight: `qwen3:8b` -- Already installed. Fits in ~6 GB. Ranked #1 for agent tool-calling in the <=8GB tier by LocalAIMaster's 2026 harness tests.

2. PDF_VISION_MODEL -- Local Vision for Screenshot Anomaly Detection

Use case: Selectively annotates failed/crashed/regression screenshots for executive PDF reports. Must run locally via Ollama. Needs strong OCR and visual anomaly detection on web UI screenshots.

Top Recommendations (Researched)

Rank	Model	Size	VRAM (Q4)	OCR Quality	Rationale
1	`qwen3-vl:30b`	30B	~19 GB	Excellent (DocVQA ~96%)	Best local VLM overall. Thinking mode for deep visual reasoning. 256K context.
2	`qwen3-vl:latest` (8B)	8B	~6.1 GB	Very Good (OCRBench ~896)	Sweet spot for 8-16GB. 15-60% faster than Qwen2.5-VL.
3	`gemma4:12b`	12B	~7.6 GB	Good	Native multimodal. Configurable visual token budget. Runs in Ollama natively.
4	`llama3.2-vision:latest`	11B	~7.8 GB	Good (DocVQA 88.4)	Current default. Battle-tested. Falling behind Qwen3-VL on benchmarks.
5	`qwen2.5vl:7b`	7B	~6.0 GB	Very Good (DocVQA 95.7)	Proven workhorse. Still strong for document OCR.

Already Installed (Relevant)

- `qwen3-vl:30b` (19 GB) -- **installed, top pick**
- `qwen3-vl:latest` (6.1 GB) -- installed
- `gemma4:12b` (7.6 GB) -- installed
- `llama3.2-vision:latest` (7.8 GB) -- installed (current default)
- `qwen2.5vl:7b` (6.0 GB) -- installed
- `llava:13b` (8.0 GB) -- installed (legacy)
- `moondream:1.8b-v2-q8_0` (2.4 GB) -- installed (lightweight)

Recommendation

Primary: `qwen3-vl:30b` -- Already installed. Qwen3-VL replaced Qwen2.5-VL as the vision model to beat in late 2025. The 30B variant delivers near-cloud quality on DocVQA (~96%), OCRBench (~920+), and GUI grounding (ScreenSpot ~92-94%). Its Thinking mode enables deep visual reasoning for complex anomaly detection -- exactly what the PDF audit pipeline needs. At 19 GB it fits comfortably in your hardware.

Fallback: `qwen3-vl:latest` **(8B)** -- Already installed. If the 30B is too slow for the PDF pipeline's throughput needs, the 8B variant is 15-60% faster than Qwen2.5-VL at equivalent quality and still outperforms llama3.2-vision on every benchmark.

Current default `llama3.2-vision` is two generations behind. Qwen3-VL 8B beats it on MathVista (85.8 vs 51.5), MMMU (58.6 vs 50.7), and DocVQA (95+ vs 88.4) despite being smaller.

---

3. VISION_MODEL -- General Vision (Currently Cloud)

Use case: Currently set to `gemini-3-flash-preview` (cloud API). This is used for general visual tasks in the agent pipeline. The research below covers local Ollama alternatives if you want to move this off-cloud.

Top Local Alternatives (Researched)

Rank	Model	Size	VRAM (Q4)	Best For	Notes
1	`qwen3-vl:30b`	30B	~19 GB	General vision + OCR + GUI	Same as PDF_VISION_MODEL recommendation
2	`gemma4:26b`	26B MoE (4B active)	~17 GB	Fast multimodal	MMMU Pro 73.8%, MATH-Vision 82.4%
3	`gemma4:31b`	31B dense	~19 GB	Highest quality	MMMU Pro 76.9%
4	`qwen3.6:27b`	27B dense	~17 GB	Native multimodal SOTA	Vision baked into base model (not bolt-on). Needs llama.cpp/LM Studio for vision -- Ollama doesn't wire mmproj yet.

Already Installed (Relevant)

- `qwen3-vl:30b` (19 GB) -- installed
- `gemma4:26b` (17 GB) -- installed
- `gemma4:31b` (19 GB) -- installed
- `qwen3.6:27b-q8_0` (29 GB) -- installed (text-only in Ollama currently)

Recommendation

If moving VISION_MODEL to local: `qwen3-vl:30b` (already installed). It's the strongest local VLM available in Ollama today. For a faster alternative, `gemma4:26b` (already installed) with its MoE architecture (3.8B active per token) provides excellent speed while maintaining MMMU Pro 73.8%.

Note: `qwen3.6:27b` has vision baked into its base architecture (not a bolt-on encoder) and scores higher on general benchmarks, but as of July 2026 Ollama does not yet wire up its mmproj sidecar -- vision input requires llama.cpp or LM Studio.

Summary: Recommended Configuration

```
# Primary recommendations (all already installed)
export OLLAMA_MODEL="qwen3.6:35b-a3b-mtp-q8_0"    # Action planning (was minimax-m3:cloud)
export PDF_VISION_MODEL="qwen3-vl:30b"              # Screenshot anomaly detection (was llama3.2-vision)
export VISION_MODEL="qwen3-vl:30b"                  # General vision (was gemini-3-flash-preview cloud)

# Fallback options (all installed)
# OLLAMA_MODEL=qwen3-coder:30b        # If 35B-A3B has tool-call issues
# OLLAMA_MODEL=qwen3:8b               # Lightweight fallback
# PDF_VISION_MODEL=qwen3-vl:latest    # Faster 8B variant
# PDF_VISION_MODEL=gemma4:12b         # Native Ollama, configurable token budget
# VISION_MODEL=gemma4:26b             # Fast MoE alternative
```

Key Changes from Current Defaults

Variable	Current	Recommended	Reason
`OLLAMA_MODEL`	`minimax-m3:cloud`	`qwen3.6:35b-a3b-mtp-q8_0`	Cloud-only -> local SOTA with dedicated tool-call parser
`PDF_VISION_MODEL`	`llama3.2-vision`	`qwen3-vl:30b`	2 generations behind -> current best local VLM
`VISION_MODEL`	`gemini-3-flash-preview`	`qwen3-vl:30b`	Cloud API -> local, no per-token cost

Models Requiring Download

None. All three recommended primary models are already in your local Ollama inventory. The only action needed is updating the environment variables / config defaults.

Sources

[LocalAIMaster: Best Ollama Models for AI Agents 2026](https://localaimaster.com/blog/best-ollama-models-for-agents)

[LocalAIMaster: Best Local LLMs for Tool & Function Calling 2026](https://localaimaster.com/blog/best-ollama-models-tool-calling)

[InsiderLLM: Best Local LLMs for Function Calling](https://insiderllm.com/guides/function-calling-local-llms/)

[InsiderLLM: Best Vision Models You Can Run Locally](https://insiderllm.com/guides/vision-models-locally/)

[PromptQuorum: Best Local Vision Models 2026](https://www.promptquorum.com/power-local-llm/local-vision-models-llava-ollama-2026)

[Morph: Best Ollama Models 2026 Ranked](https://www.morphllm.com/best-ollama-models)

[SumGuy: Local Vision LLMs Worth Running in 2026](https://sumguy.com/multimodal-llms-pixtral-llava-qwen-vl/)

[ServerMan: Best Ollama Vision Models 2026](https://www.serverman.co.uk/ai/ollama/best-ollama-models-for-vision/)